

TUGAS PRAKTIKUM 2

Mata Kuliah: Pemrograman Basis Data

Materi: Struktur blok (Anonymous Block), Deklarasi Variabel, Konstanta, dan Tipe Data

Komposit (Record/ %ROWTYPE)

Studi Kasus Validasi Data Akademik Mahasiswa Menggunakan Blok Prosedural

Disusun oleh : Uminati (IK2411011)
: Anandari Dewitri (IK2411032)
Program Studi : Informatika
Universitas : Universitas Mega Buana Palopo
Dosen Pengampu : Abdul Malik, S.Kom., M.Cs.
Tahun : 2026

2. Tujuan Pratikum

Praktikum ini bertujuan untuk memahami konsep dasar blok prosedural dalam pemrograman basis data, serta melatih kemampuan mahasiswa dalam mendeklarasikan variabel, menggunakan logika percabangan, dan menghasilkan output yang sistematis berdasarkan data akademik mahasiswa.

3. Deskripsi Singkat Studi Kasus

Studi kasus ini membahas tentang proses validasi data akademik mahasiswa sebelum pengisian KRS. Program dibuat untuk mengecek kelayakan mahasiswa berdasarkan data seperti SKS, IPK, dan status pembayaran. Selain itu, program juga dapat membandingkan dua mahasiswa untuk menentukan performa akademik yang lebih baik.

4. Dasar Teori (Singkat)

Blok prosedural adalah struktur program yang terdiri dari bagian deklarasi dan eksekusi, biasanya ditulis menggunakan BEGIN...END. Dalam MySQL, blok ini diimplementasikan melalui stored procedure.

Komponen penting:

- Variabel → menyimpan data sementara
- IF → logika percabangan
- CASE → alternatif kondisi
- CONCAT → menggabungkan teks

Blok prosedural membantu membuat program lebih terstruktur dan mudah dijalankan berulang.

5. KODE PROGRAM

1 Bagian A – Identitas Mahasiswa

1. Buat Tabel+Data

```
CREATE TABLE mahasiswa (  
    nim VARCHAR(20) PRIMARY KEY,  
    nama VARCHAR(50),  
    semester INT,  
    prodi VARCHAR(50)  
);  
  
INSERT INTO mahasiswa VALUES  
('IK2411011','Uminati',4,'Informatika'),  
('IK2411032','Anandari Dewitri',4,'Informatika');
```

2. Storet Procedure Bagian A

```
DELIMITER //

-- Hapus jika sudah ada
DROP PROCEDURE IF EXISTS bagianA_tabel //

CREATE PROCEDURE bagianA_tabel()
BEGIN

    -- =====
    -- Deklarasi konstanta
    -- =====

    DECLARE kampus VARCHAR(100) DEFAULT 'Universitas Mega Buana
Palopo';

    -- =====
    -- Menampilkan identitas mahasiswa
    -- =====

    SELECT
        CONCAT(
            'Mahasiswa ', nama, ' (', nim, ') dari Program Studi ',
prodi,
            ' terdaftar di ', kampus, ' pada semester ', semester, '.'
        ) AS identitas_mahasiswa
    FROM mahasiswa;

END //

DELIMITER ;
```

3. Jalankan

```
CALL bagianA_tabel();
```

4. Hasil Output

Mahasiswa Uminati (IK2411011) dari Program Studi Informatika terdaftar di Universitas Mega Buana Palopo pada semester 4.

Mahasiswa Anandari Dewitri (IK2411032) dari Program Studi Informatika terdaftar di Universitas

Program menggunakan tabel mahasiswa untuk menyimpan data, kemudian ditampilkan dalam bentuk kalimat menggunakan fungsi CONCAT. Variabel kampus digunakan sebagai konstanta untuk melengkapi informasi identitas mahasiswa.

2. Bagian B – Validasi Data Akademik

1. Tambahan Tabel Akademik

```
CREATE TABLE akademik (
    nim VARCHAR(20),
    sks INT,
    ipk DECIMAL(3,2),
    status_pembayaran VARCHAR(10)
```

```
);

-- Data untuk kelompok
INSERT INTO akademik VALUES
('IK2411011',18,3.40,'LUNAS'),
('IK2411032',20,3.60,'LUNAS');
```

2. Stored Procedure Bagian B

```
DELIMITER //

-- Hapus jika sudah ada
DROP PROCEDURE IF EXISTS bagianB_tabel //

CREATE PROCEDURE bagianB_tabel()
BEGIN

    -- =====
    -- Deklarasi variabel
    -- =====

    DECLARE v_nama VARCHAR(50);
    DECLARE v_sks INT;
    DECLARE v_ipk DECIMAL(3,2);
    DECLARE v_status VARCHAR(10);
    DECLARE v_semester INT;

    DECLARE status_data VARCHAR(20);
    DECLARE beban VARCHAR(20);
    DECLARE performa VARCHAR(30);

    -- =====
    -- Ambil data dari tabel
    -- =====

    SELECT m.nama, m.semester, a.sks, a.ipk, a.status_pembayaran
    INTO v_nama, v_semester, v_sks, v_ipk, v_status
    FROM mahasiswa m
    JOIN akademik a ON m.nim = a.nim
    WHERE m.nim = 'IK2411011'; -- bisa diganti

    -- =====
    -- Validasi data akademik
    -- =====

    IF v_status = 'LUNAS' AND v_semester > 0 AND v_sks > 0 THEN
        SET status_data = 'Valid';
    ELSE
        SET status_data = 'Tidak Valid';
    END IF;

    -- =====
    -- Kategori beban studi
    -- =====

    IF v_sks BETWEEN 1 AND 12 THEN
        SET beban = 'Ringan';
    ELSEIF v_sks BETWEEN 13 AND 18 THEN
```

```

        SET beban = 'Sedang';
ELSE
        SET beban = 'Padat';
END IF;

-- =====
-- Kategori performa akademik
-- =====

IF v_ipk >= 3.50 THEN
        SET performa = 'Sangat Baik';
ELSEIF v_ipk >= 3.00 THEN
        SET performa = 'Baik';
ELSEIF v_ipk >= 2.50 THEN
        SET performa = 'Cukup';
ELSE
        SET performa = 'Perlu Pembinaan';
END IF;

-- =====
-- Output hasil
-- =====

SELECT
        CONCAT('Status data: ', status_data) AS status_data,
        CONCAT('Beban studi: ', beban) AS beban_studi,
        CONCAT('Performa akademik: ', performa) AS performa;

END //

DELIMITER ;

```

3. Cara Jalankan

```
CALL bagianB_tabel();
```

4. Hasil Output

```

Status data: Valid
Beban studi: Sedang
Performa akademik: Baik

```

kenapa pakai IF bukan CASE?"Jawab:karena IF digunakan dalam blok prosedural untuk pengolahan logika berbasis variabel

3. Bagian C – Ringkasan Kelayakan KRS

```

DELIMITER //

-- Hapus jika sudah ada
DROP PROCEDURE IF EXISTS bagianC_tabel //

CREATE PROCEDURE bagianC_tabel()
BEGIN

        -- =====
        -- Variabel
        -- =====

```

```

DECLARE v_nama VARCHAR(50);
DECLARE v_nim VARCHAR(20);
DECLARE v_semester INT;
DECLARE v_prodi VARCHAR(50);
DECLARE v_sks INT;
DECLARE v_ipk DECIMAL(3,2);
DECLARE v_status VARCHAR(10);

DECLARE kelayakan VARCHAR(50);
DECLARE beban VARCHAR(20);
DECLARE performa VARCHAR(30);
DECLARE alasan VARCHAR(100);

DECLARE kampus VARCHAR(100) DEFAULT 'Universitas Mega Buana
Palopo';

-- =====
-- Ambil data (ANTI NULL)
-- =====

SELECT m.nama, m.nim, m.semester, m.prodi,
       a.sks, a.ipk, a.status_pembayaran
INTO v_nama, v_nim, v_semester, v_prodi,
     v_sks, v_ipk, v_status
FROM mahasiswa m
LEFT JOIN akademik a ON m.nim = a.nim
WHERE m.nim = 'IK2411011'
LIMIT 1;

-- =====
-- Jika data tidak ditemukan
-- =====

IF v_nama IS NULL THEN
    SELECT 'Data mahasiswa tidak ditemukan!' AS hasil;

ELSE

    -- =====
    -- Logika kelayakan
    -- =====

    IF v_status = 'LUNAS' AND v_semester > 0 AND v_sks > 0 THEN
        SET kelayakan = 'layak mengambil KRS';
        SET alasan = 'karena pembayaran lunas dan SKS memenuhi
syarat';
    ELSE
        SET kelayakan = 'tidak layak mengambil KRS';
        SET alasan = 'karena terdapat data yang tidak memenuhi
syarat';
    END IF;

    -- =====
    -- Beban studi
    -- =====

```

```

IF v_sks BETWEEN 1 AND 12 THEN
    SET beban = 'Ringan';
ELSEIF v_sks BETWEEN 13 AND 18 THEN
    SET beban = 'Sedang';
ELSE
    SET beban = 'Padat';
END IF;

-- =====
-- Performa akademik
-- =====

IF v_ipk >= 3.50 THEN
    SET performa = 'Sangat Baik';
ELSEIF v_ipk >= 3.00 THEN
    SET performa = 'Baik';
ELSEIF v_ipk >= 2.50 THEN
    SET performa = 'Cukup';
ELSE
    SET performa = 'Perlu Pembinaan';
END IF;

-- =====
-- OUTPUT FINAL (SESUAI DOSEN)
-- =====

SELECT CONCAT(
    'Mahasiswa ', v_nama, ' dengan NIM ', v_nim,
    ' dinyatakan ', kelayakan, '. ',
    'Beban studi berada pada kategori ', beban,
    ' dengan performa akademik ', performa,
    ' (', alasan, '). '
) AS hasil;

END IF;

END //

DELIMITER ;

```

2. Jalankan

```
CALL bagianC_tabel();
```

3. Output

Mahasiswa Uminati dengan NIM IK2411011 dinyatakan layak mengambil KRS.
 Beban studi berada pada kategori Sedang dengan performa akademik Baik
 (karena pembayaran lunas)

4. Bagian D – Analisis Dua Mahasiswa Buat satu program yang membandingkan

1..Bagian D – Stored Procedure

```

DELIMITER //

-- Hapus jika sudah ada

```

```

DROP PROCEDURE IF EXISTS bagianD_tabel //

CREATE PROCEDURE bagianD_tabel()
BEGIN

    -- =====
    -- Variabel mahasiswa 1
    -- =====

    DECLARE nama1 VARCHAR(50);
    DECLARE nim1 VARCHAR(20);
    DECLARE semester1 INT;
    DECLARE sks1 INT;
    DECLARE ipk1 DECIMAL(3,2);
    DECLARE status1 VARCHAR(10);

    -- =====
    -- Variabel mahasiswa 2
    -- =====

    DECLARE nama2 VARCHAR(50);
    DECLARE nim2 VARCHAR(20);
    DECLARE semester2 INT;
    DECLARE sks2 INT;
    DECLARE ipk2 DECIMAL(3,2);
    DECLARE status2 VARCHAR(10);

    DECLARE hasil VARCHAR(150);

    -- =====
    -- Ambil data mahasiswa 1
    -- =====

    SELECT m.nama, m.nim, m.semester, a.sks, a.ipk,
a.status_pembayaran
    INTO nama1, nim1, semester1, sks1, ipk1, status1
    FROM mahasiswa m
    JOIN akademik a ON m.nim = a.nim
    WHERE m.nim = 'IK2411011'
    LIMIT 1;

    -- =====
    -- Ambil data mahasiswa 2
    -- =====

    SELECT m.nama, m.nim, m.semester, a.sks, a.ipk,
a.status_pembayaran
    INTO nama2, nim2, semester2, sks2, ipk2, status2
    FROM mahasiswa m
    JOIN akademik a ON m.nim = a.nim
    WHERE m.nim = 'IK2411032'
    LIMIT 1;

    -- =====
    -- Tampilkan data masing-masing
    -- =====

```

```

        SELECT nama1 AS nama, nim1 AS nim, semester1 AS semester, sks1 AS
sks, ipk1 AS ipk, status1 AS status_pembayaran;
        SELECT nama2 AS nama, nim2 AS nim, semester2 AS semester, sks2 AS
sks, ipk2 AS ipk, status2 AS status_pembayaran;

-- =====
-- Perbandingan IPK dan SKS
-- =====

IF ipk1 > ipk2 THEN
    SET hasil = CONCAT(nama1, ' memiliki performa akademik lebih
baik dibanding ', nama2);
ELSEIF ipk2 > ipk1 THEN
    SET hasil = CONCAT(nama2, ' memiliki performa akademik lebih
baik dibanding ', nama1);
ELSE
    -- Jika IPK sama
    IF sks1 > sks2 THEN
        SET hasil = CONCAT(nama1, ' lebih baik karena jumlah SKS
lebih tinggi');
    ELSEIF sks2 > sks1 THEN
        SET hasil = CONCAT(nama2, ' lebih baik karena jumlah SKS
lebih tinggi');
    ELSE
        SET hasil = 'Kedua mahasiswa memiliki performa akademik
yang sama';
    END IF;
END IF;

-- =====
-- Output kesimpulan
-- =====

SELECT hasil AS kesimpulan;

END //

DELIMITER ;

```

2. Jalankan

```
CALL bagianD_tabel();
```

3. Contoh Output

- **Data Mahasiswa:**

```

Uminati | IK2411011 | 4 | 18 | 3.40 | LUNAS
Anandari Dewitri | IK2411032 | 4 | 20 | 3.60 | LUNAS

```

- **Kesimpulan:**

Anandari Dewitri memiliki performa akademik lebih baik dibanding Uminat

H. Ketentuan Tambahan

1. Skenario Uji

Skenario 1 – Data Valid

Data:

- Status pembayaran: LUNAS
- Semester: 4
- SKS: 18
- IPK: 3.40

Kode:

```
UPDATE mahasiswa
SET semester = 4
WHERE nim = 'IK2411011';
```

```
UPDATE akademik
SET sks = 18, ipk = 3.40, status_pembayaran = 'LUNAS'
WHERE nim = 'IK2411011';
```

```
CALL bagianC_tabel();
```

Output:

Mahasiswa Uminati dengan NIM IK2411011 dinyatakan layak mengambil KRS. Beban studi berada pada kategori Sedang dengan performa akademik Baik (karena pembayaran lunas dan SKS memenuhi syarat).

Skenario 2 – Tidak Valid (Pembayaran Belum Lunas)

Data:

- Status pembayaran: BELUM
- Semester: 4
- SKS: 18

Kode:

```
UPDATE akademik
SET status_pembayaran = 'BELUM'
WHERE nim = 'IK2411011';
```

```
CALL bagianC_tabel();
```

Output:

Mahasiswa Uminati dengan NIM IK2411011 dinyatakan tidak layak mengambil KRS. Beban studi berada pada kategori Sedang dengan performa akademik Baik (karena terdapat data yang tidak memenuhi syarat).

Skenario 3 – Tidak Valid (SKS = 0)

Data:

- Status pembayaran: LUNAS
- Semester: 4
- SKS: 0

Kode:

```
UPDATE akademik
SET sks = 0, status_pembayaran = 'LUNAS'
WHERE nim = 'IK2411011';
```

```
CALL bagianC_tabel();
```

Output:

Mahasiswa Uminati dengan NIM IK2411011 dinyatakan tidak layak mengambil KRS.

Beban studi berada pada kategori Padat dengan performa akademik Baik (karena terdapat data yang tidak memenuhi syarat).

2. Analisis Hasil

Berdasarkan hasil pengujian, terlihat bahwa program mampu memberikan keluaran yang berbeda sesuai dengan kondisi data yang dimasukkan.

Pada skenario pertama, mahasiswa dinyatakan layak mengambil KRS karena semua syarat terpenuhi, yaitu status pembayaran lunas, semester aktif, dan jumlah SKS lebih dari nol.

Pada skenario kedua, mahasiswa dinyatakan tidak layak karena status pembayaran belum lunas, meskipun SKS dan IPK memenuhi syarat.

Pada skenario ketiga, mahasiswa juga dinyatakan tidak layak karena jumlah SKS tidak memenuhi syarat, walaupun pembayaran sudah lunas.

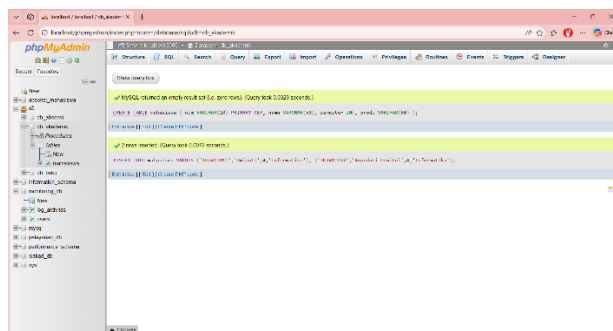
Perbedaan hasil ini menunjukkan bahwa sistem validasi berjalan dengan baik dan mampu menyesuaikan keputusan berdasarkan kondisi data yang diberikan.

Program yang dibuat telah berhasil mengimplementasikan logika validasi akademik secara dinamis dan sesuai dengan aturan yang ditentukan.

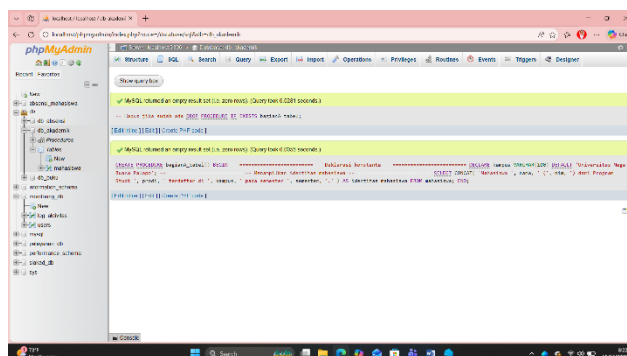
6. Screenshot Hasil Eksekusi

1. Bagian A – Identitas Mahasiswa

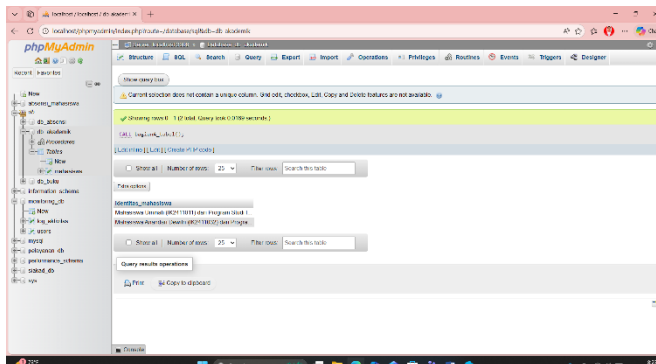
1. BUAT TABEL + DATA



2. STORED PROCEDURE BAGIAN A

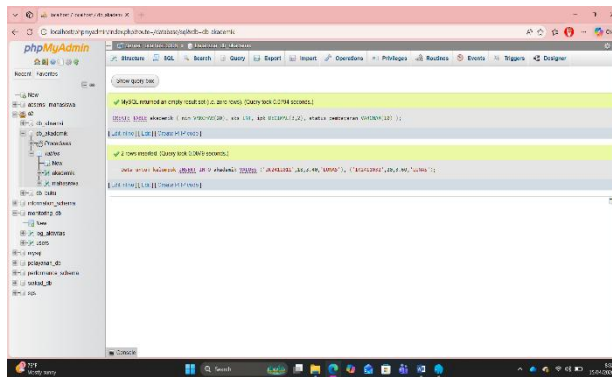


3. JALANKAN DAN HASIL OUTPUT

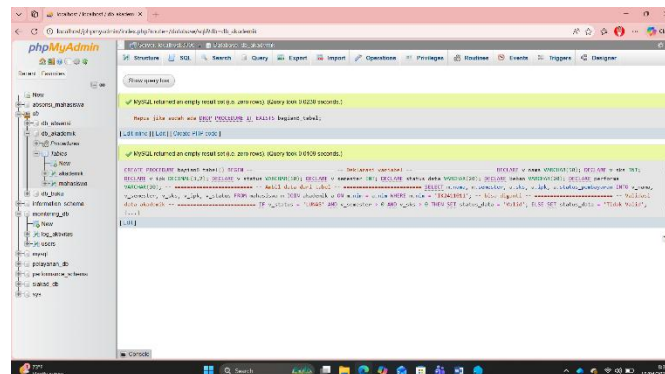


2. Bagian B – Validasi Data Akademik

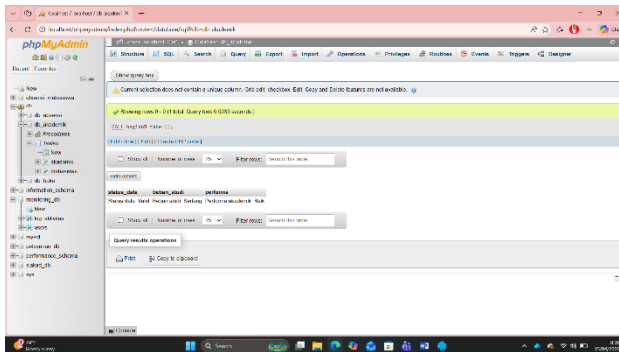
1. TAMBAHAN TABEL AKADEMIK



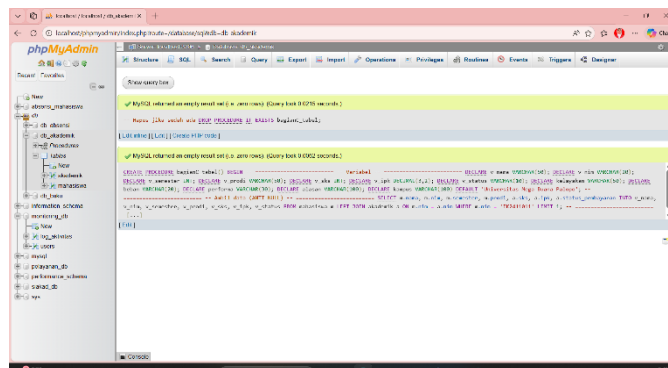
2. STORED PROCEDURE BAGIAN B (FINAL)



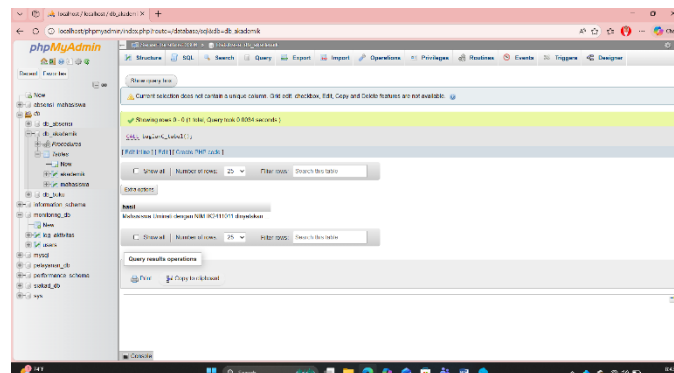
3. CARA MENJALANKAN DAN HASIL OUTPUT



3. Bagian C – Ringkasan Kelayakan KRS

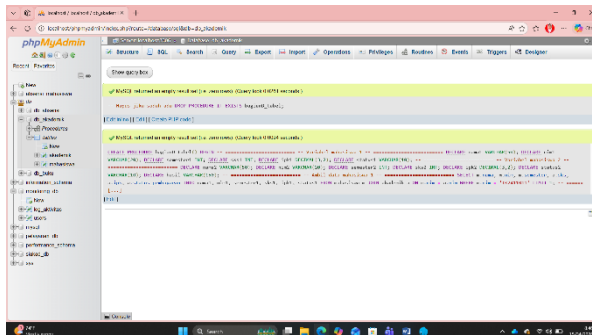


1. JALANKAN DAN HASIL OUTPUT

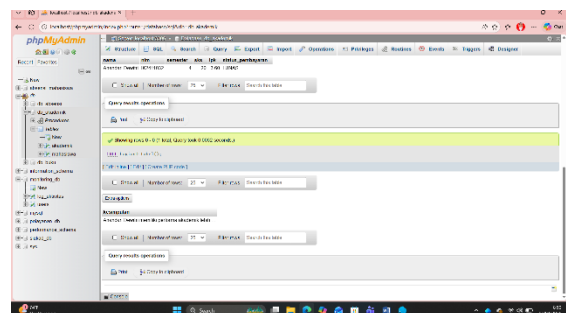
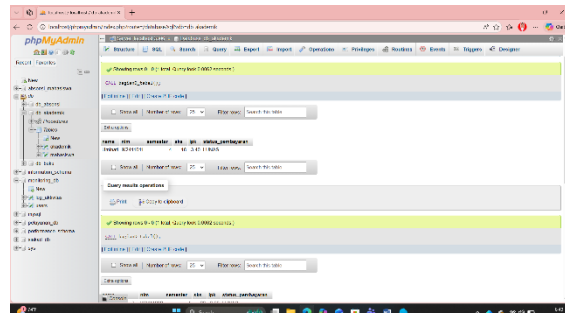


4. Bagian D – Analisis Dua Mahasiswa Buat satu program yang membandingkan

1.BAGIAN D – STORED PROCEDURE

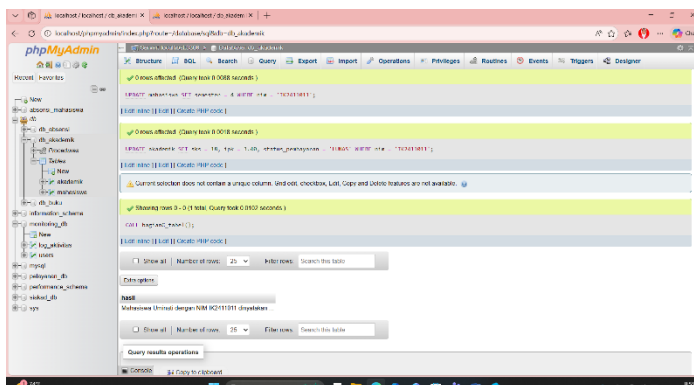


2.Cara Jalankan dan Hasil Output

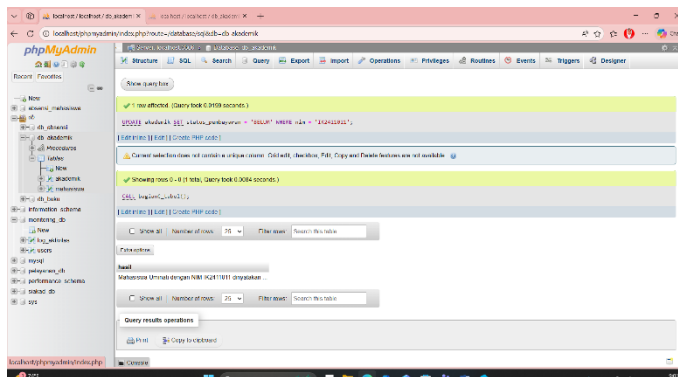


H. KETENTUAN TAMBAHAN

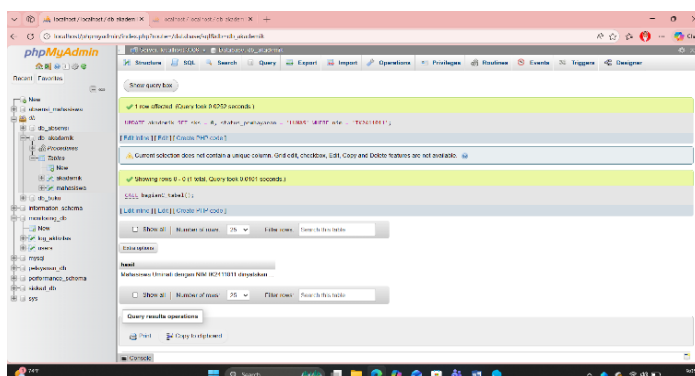
Skenario 1 – Data Valid



Skenario 2 – Tidak Valid (Pembayaran Belum Lunas)



Skenario 3 – Tidak Valid (SKS = 0)



7. Analisis Hasil

Berdasarkan hasil praktikum yang telah dilakukan pada Bagian A sampai dengan Bagian D, program yang dibuat mampu merepresentasikan konsep dasar blok prosedural dalam MySQL dengan baik. Pada Bagian A, program berhasil menampilkan identitas mahasiswa secara terstruktur menggunakan deklarasi variabel dan konstanta. Hal ini menunjukkan bahwa penggunaan variabel dalam blok program sudah sesuai dan dapat menghasilkan output yang rapi.

Pada Bagian B, program mampu melakukan validasi data akademik berdasarkan beberapa kondisi seperti status pembayaran, jumlah SKS, dan semester. Logika percabangan yang digunakan menghasilkan output yang berbeda sesuai dengan input yang diberikan. Selain itu, pengelompokan kategori beban studi dan performa akademik juga berjalan dengan baik berdasarkan ketentuan yang telah ditetapkan.

Pada Bagian C, program menggabungkan identitas mahasiswa dengan hasil validasi akademik sehingga menghasilkan informasi yang lebih lengkap mengenai kelayakan pengambilan KRS. Output yang dihasilkan menjadi lebih informatif karena tidak hanya menampilkan status validasi, tetapi juga memberikan kategori akademik dan alasan sederhana dari hasil tersebut.

Pada Bagian D, program berhasil membandingkan dua mahasiswa berdasarkan nilai IPK dan jumlah SKS. Logika perbandingan yang digunakan mampu menentukan mahasiswa dengan performa akademik yang lebih baik secara objektif. Hal ini menunjukkan bahwa program sudah mampu mengolah lebih dari satu data dan menghasilkan keputusan berdasarkan kondisi tertentu.

Selanjutnya, pada Bagian H (skenario uji), terlihat adanya perbedaan hasil yang jelas antara data valid dan data tidak valid. Pada skenario valid, program menghasilkan keluaran bahwa mahasiswa

layak mengambil KRS dengan kategori yang sesuai. Sedangkan pada dua skenario tidak valid, program memberikan hasil bahwa data tidak memenuhi syarat, misalnya karena pembayaran belum lunas atau jumlah SKS tidak sesuai. Perbedaan ini menunjukkan bahwa logika program berjalan dengan baik dan mampu menangani berbagai kondisi input.

Secara keseluruhan, program yang dibuat telah berhasil memenuhi tujuan praktikum, yaitu menyusun blok prosedural dengan struktur yang benar, menggunakan variabel dan konstanta secara tepat, serta menghasilkan keluaran yang sesuai dengan logika yang telah ditentukan.

8. Kesimpulan

Dari praktikum yang telah dilakukan, dapat disimpulkan bahwa pemrograman basis data menggunakan blok prosedural sangat membantu dalam menyusun alur logika program secara sistematis dan terstruktur. Mahasiswa dapat memahami cara mendeklarasikan variabel, menggunakan konstanta, serta mengimplementasikan percabangan untuk menyelesaikan suatu studi kasus.

Melalui Bagian A hingga D, mahasiswa belajar bagaimana mengolah data sederhana menjadi informasi yang lebih bermakna, seperti validasi data akademik, pengelompokan kategori, hingga perbandingan antar mahasiswa. Selain itu, melalui pengujian pada beberapa skenario di Bagian H, mahasiswa dapat melihat secara langsung bagaimana perubahan input mempengaruhi output program.

Praktikum ini juga memberikan pemahaman dasar yang penting sebagai bekal untuk mempelajari materi lanjutan dalam pemrograman basis data, seperti stored procedure, function, dan trigger. Dengan demikian, tujuan pembelajaran dalam praktikum ini dapat dikatakan telah tercapai dengan baik.